CMSC 12 Cheat Sheet: Sequential, Conditional, Iterative

Tutor copy. Session with Alexandra and Ernstine, Mon 21 Sep 2026, 2 pm, Main Lib. Prof. Albacea section. Plan: pre-test 5, discuss 10, guided 25, post-test 15.

Sequential

```
r = float(input("Enter the radius: "))
h = float(input("Enter the height: "))
pi = 3.14
area_circ = pi * r * r
circumference = 2 * pi * r
area_rect = circumference * h
sa_cyl = area_rect + 2 * area_circ
print("The surface area is", sa_cyl)
```

- `input()` returns str. Typecast: `int()`, `float()`.
- `=` assigns, `==` compares.
- `print(a, b)` space-separated. `end=""` no newline. `print()` newline.
- `/` float division, `//` integer division, `%` remainder, `**` power.

Conditional

```
n = float(input("Enter a number: "))
if n < 0:
    print("Negative")
elif n > 0:
    print("Positive")
else:
    print("Zero")
```

```
n = int(input("Enter an integer: "))
if n % 2 == 0:
    if n % 3 == 0:
        print("divisible by 2 and 3")
    else:
        print("divisible by 2 only")
elif n % 3 == 0:
    print("divisible by 3 only")
else:
    print("not divisible by 2 nor 3")
```

- One branch runs per chain. Order matters.
- `and`, `or`, `not`. Compare: `< <= > >= == !=`.
- Indentation is the block. 4 spaces.

Iterative

```
n = int(input("Enter an integer: "))
i = n
while i > 0:        # init, test, update
    print(i)
    i = i - 1

for i in range(n, 0, -1):
    print(i)
```

range	values
range(5)	0 1 2 3 4
range(1, 6)	1 2 3 4 5
range(5, 0, -1)	5 4 3 2 1
range(0, 10, 2)	0 2 4 6 8

```
# x^y by repeated multiplication
product = x
for counter in range(1, y):
    product = product * x

# guess the number: loop until correct
import random
r = random.randint(1, 100)
while True:
    g = int(input("Guess: "))
    if g == r:
        print("Correct!")
        break
    elif g > r:
        print("Lower!")
    else:
        print("Higher!")

# square root, guess and check
error = 0.01
g = x / 2
while g*g > x + error or g*g < x - error:
    g = (g + x / g) / 2
```

- while: unknown count. for: known count.
- Forgot the update line = infinite loop.
- `break` exits loop. `while True` + `break` = repeat until.

Pattern printing (nested loops)

Recipe: draw n = 4 on paper. Per row i (from 0), count spaces and chars. Find formulas. Outer loop = rows, inner loops = one row, `print()` ends row.

```
# filled square n x n
for i in range(n):
    for j in range(n):
        print("X", end="")
    print()

# hollow square
for i in range(n):
    for j in range(n):
        if i == 0 or j == 0 or i == n-1 or j == n-1:
            print("X", end="")
        else:
            print(" ", end="")
    print()

# right triangle, height n
```

```
# equilateral (centered) triangle, height n
# row i: spaces = n-1-i, stars = 2i+1
for i in range(n):
    for s in range(n - 1 - i):
        print(" ", end="")
    for k in range(2 * i + 1):
        print("*", end="")
    print()

# same, string repeat shortcut
for i in range(n):
    print(" " * (n-1-i) + "*" * (2*i+1))

# upside down: range(n-1, -1, -1)
# box L wide, W tall: outer range(W), inner range(L)
# christmas tree: repeat the triangle
```

```
for i in range(n):
    for j in range(i + 1):
        print("*", end="")
    print()
```

row i	0	1	2	3
spaces (n=4)	3	2	1	0
stars	1	3	5	7

```
#   n times, each one wider, same width base
#   (pad spaces using the final width)
```

Pre-test answers

1. TypeError: str + int. Fix with `int(input())`.
2. both
3. `5 4 3 2 1`
4. `*` / `**` / `***` on three lines.
5. 16. Body runs 3 times (counter = 1, 2, 3).

Guided problem solutions

A. Divisors, prime (ex. 1, 6)

```
n = int(input("Enter n: "))
count = 0
for d in range(1, n + 1):
    if n % d == 0:
        print(d)
        count = count + 1
if count == 2:
    print("prime")
else:
    print("composite")

# early exit version
is_prime = n > 1
for d in range(2, n):
    if n % d == 0:
        is_prime = False
        break
```

Questions to ask: range of candidate divisors (1 to n), test is `n % d == 0`, counter starts at 0 before the loop, prime has exactly 2 divisors.

B. Equilateral triangle (ex. 10)

See pattern section. Push them to fill the table (spaces, stars per row) before typing. Upside down: reverse the outer range. Extension: box (ex. 11), hollow box, tree (ex. 12).

C. GCF, LCM (ex. 3, 4)

```
a = int(input("A: "))
b = int(input("B: "))
gcf = 1
for d in range(1, min(a, b) + 1):
    if a % d == 0 and b % d == 0:
        gcf = d          # keeps the biggest
print("GCF:", gcf)

# count-down version with break
d = min(a, b)
while d > 0:
    if a % d == 0 and b % d == 0:
        break
    d = d - 1

# LCM by counting multiples of a
m = a
while m % b != 0:
    m = m + a
print("LCM:", m)
# or: lcm = a * b // gcf
# LCM of three (ex. 5): lcm(lcm(a,b), c)
```

If they have not seen `min()`: use an if to pick the smaller.

Post-test solutions

```
# 1. n x n multiplication table (ex. 9)
n = int(input("n: "))
for i in range(1, n + 1):
    for j in range(1, n + 1):
        print(i * j, end="\t")
    print()

# 2. count primes between n and 2n (ex. 7)
n = int(input("n: "))
count = 0
for x in range(n, 2 * n + 1):
    if x < 2:
        continue
    is_prime = True
    for d in range(2, x):
        if x % d == 0:
            is_prime = False
            break
    if is_prime:
```

```
# 3. hollow box L wide, W tall (ex. 11)
L = int(input("L: "))
W = int(input("W: "))
for i in range(W):
    for j in range(L):
        if i == 0 or i == W-1 or j == 0 or j == L-1:
            print("X", end="")
        else:
            print(" ", end="")
    print()

# extra: count zeroes (ex. 2)
n = int(input("n: "))
zeroes = 0
for k in range(n):
    v = int(input())
    if v == 0:
        zeroes = zeroes + 1
print(zeroes)
```

```
        count = count + 1
print(count)
```

```
# extra: first 30 twin primes (ex. 8)
# loop p from 2 upward, test p and p+2 prime,
# count pairs, stop at 30
```

Common student errors to watch for

- Forgetting `int()` on input, then comparing str to int.
- Off by one: `range(1, n)` skips n. `range(n)` starts at 0.
- Missing `print()` after the inner loop, so the whole shape is one line.
- Update line inside the wrong indentation, or missing, so while never ends.
- Using = inside `if`.
- Resetting the counter inside the loop instead of before it.

Sources: Abriol-Santos slides (Sequential, Conditional, Iterative Statements), Samaniego sample exercises. Session page: tutor.stimmie.dev/cmsc12